

NOM :	Prénom :	Classe : Groupe ▾
-------	----------	--

Consulter les journaux du serveur

Objectifs

Partie 1: Lire des fichiers journaux à l'aide des programmes Cat, More, Less et Tail

Partie 2: Fichiers journaux et Syslog

Partie 3: Fichiers journaux et Journalctl

Contexte/scénario

Les fichiers journaux sont extrêmement utiles pour dépanner et surveiller les réseaux. Chaque application génère des fichiers journaux différents, avec des champs et des informations qui leur sont propres. Si la structure des champs peut varier selon les fichiers journaux, les outils utilisés pour les lire sont généralement les mêmes. Au cours de ces travaux pratiques, vous allez découvrir les outils couramment utilisés pour lire les fichiers journaux et apprendre à les utiliser.

Ressources requises

= Machine virtuelle du Security Workstation (poste de travail sécurisé)

Instructions

Partie 1: Lecture des fichiers journaux avec Cat, More, Less et Tail

Les fichiers journaux enregistrent des événements spécifiques déclenchés par les applications, les services ou le système d'exploitation lui-même. Généralement stockés au format de texte clair, les fichiers journaux constituent une ressource indispensable aux opérations de dépannage.

Étape 1: Ouverture des fichiers journaux.

Les fichiers journaux contiennent généralement des informations en texte clair qui peuvent être affichées par quasiment tous les programmes capables de gérer du texte (des éditeurs de texte, par exemple). Toutefois, pour des raisons de commodité, de simplicité d'utilisation et de vitesse, certains outils sont davantage utilisés que d'autres. Cette section se concentre sur quatre programmes en ligne de commande: **cat**, **more**, **less** et **tail**.

cat, dérivé du mot "concaténer", est un outil UNIX en ligne de commande utilisé pour lire et afficher le contenu d'un fichier à l'écran. En raison de sa simplicité et de sa capacité à ouvrir un fichier texte et à l'afficher dans un terminal en mode texte, **cat** est largement utilisé à ce jour.

- a. Démarrez la **VM Security Workstation** (machine virtuelle Security Workstation) et ouvrez une fenêtre de terminal.

Capture d'écran de votre connexion à la VM Security Workstation :

```

Terminal
File Edit View Search Terminal Help
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.
[analyst@secOps ~]$

```

- b. Dans la fenêtre du terminal, exécutez la commande suivante pour afficher le contenu du fichier **logstash-tutorial.log**, situé dans le dossier **/home/analyst/lab.support.files/**:

```
analyst@secOps ~$ cat /home/analyst/lab.support.files/logstash-tutorial.log
```

Le contenu du fichier doit défiler dans la fenêtre du terminal, jusqu'à ce que tout le texte soit affiché.

Quel est l'inconvénient d'utiliser **cat** avec des fichiers texte volumineux ?

L'inconvénient d'utiliser **cat** avec des fichiers texte volumineux est qu'il affiche **tout le contenu du fichier en une seule fois sans pagination**, ce qui fait défiler rapidement les premières lignes hors de l'écran du terminal.

- On se retrouve uniquement avec la **fin du fichier** affichée.
- Il devient alors **très difficile, voire impossible, de lire le début** du fichier sans utiliser un outil supplémentaire (comme **less** ou **more**, ou en pipant la sortie vers ces outils).

Un autre outil populaire pour visualiser les fichiers journaux est **more**. Comme **cat**, **more** est un outil de ligne de commande UNIX qui peut ouvrir un fichier texte et afficher son contenu à l'écran. La principale différence entre **cat** et **more** est que **more** prend en charge les sauts de page, permettant à l'utilisateur de visualiser le contenu d'un fichier, Il suffit d'appuyer sur la barre d'espace pour afficher la page suivante.

- c. À partir de la même fenêtre de terminal, utilisez la commande ci-dessous pour afficher à nouveau le contenu du fichier **logstash-tutorial.log**. Cette fois-ci en utilisant **more**:

```
analyst@secOps ~$ more /home/analyst/lab.support.files/logstash-tutorial.log
```

Le contenu du fichier doit défiler dans la fenêtre du terminal jusqu'à ce que la première page soit affichée. Appuyez sur la barre d'espace pour passer à la page suivante. Appuyez sur Entrée pour afficher la ligne de texte suivante.

Quel est l'inconvénient d'utiliser **more**?

L'inconvénient principal d'utiliser la commande **more** est qu'elle ne permet de **faire défiler le contenu que vers l'avant (vers le bas)**.

- Une fois que vous avez avancé vers la page suivante, vous **ne pouvez pas revenir en arrière** pour visualiser les lignes ou les pages précédemment affichées.

- Ceci est une limitation par rapport à la commande **less**, qui permet la navigation bidirectionnelle (vers l'avant et vers l'arrière).

S'appuyant sur la fonctionnalité de **cat** et **more**, l'outil **less** permet d'afficher le contenu d'un fichier page par page, tout en laissant à l'utilisateur le choix de visualiser les pages précédemment affichées.

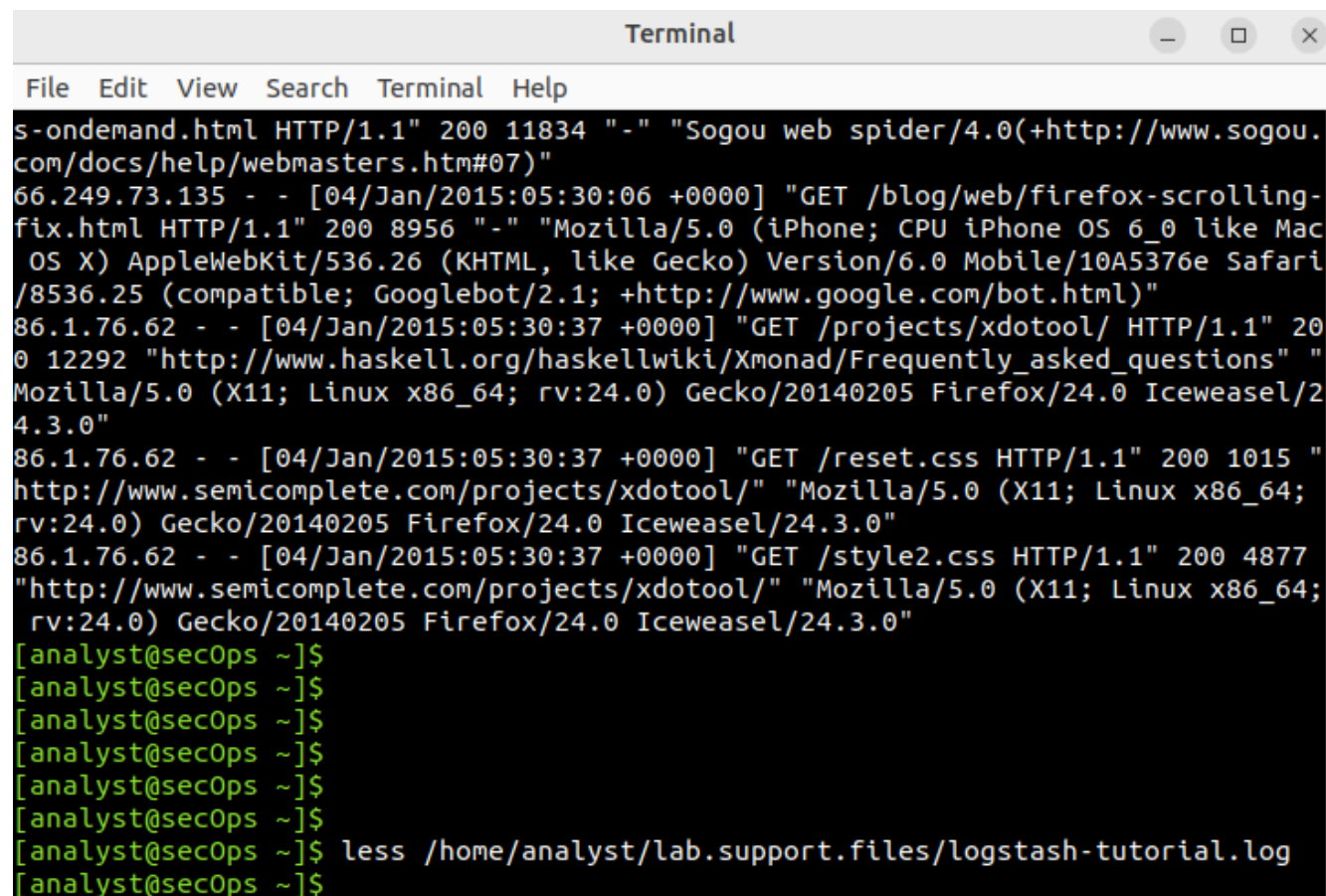
- d. Dans la même fenêtre de terminal, utilisez **less** pour afficher à nouveau le contenu du fichier **logstash-tutorial.log**:

```
analyst@secOps ~$ less /home/analyst/lab.support.files/logstash-tutorial.log
```

Le contenu du fichier doit défiler dans la fenêtre du terminal jusqu'à ce que la première page soit affichée. Appuyez sur la barre d'espace pour passer à la page suivante. Appuyez sur Entrée pour afficher la ligne de texte suivante. Utilisez les flèches haut et bas pour accéder aux différentes pages du fichier texte.

Utilisez la touche **"q"** de votre clavier pour fermer l'outil **less**.

Capture d'écran de l'affichage après l'exécution de la commande « less » :



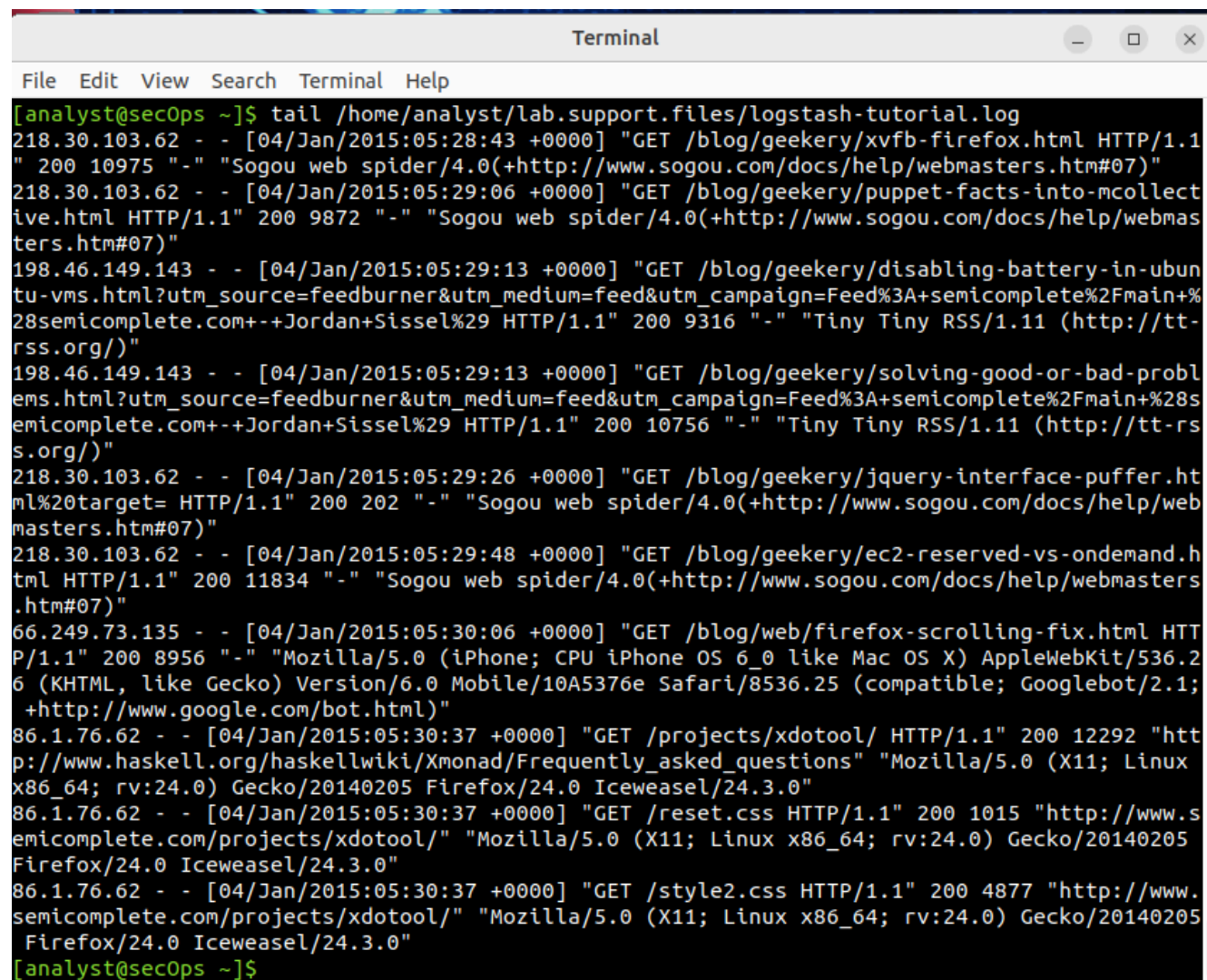
```
Terminal
File Edit View Search Terminal Help
s-ondemand.html HTTP/1.1" 200 11834 "-" "Sogou web spider/4.0(+http://www.sogou.
com/docs/help/webmasters.htm#07)"
66.249.73.135 - - [04/Jan/2015:05:30:06 +0000] "GET /blog/web/firefox-scrolling-
fix.html HTTP/1.1" 200 8956 "-" "Mozilla/5.0 (iPhone; CPU iPhone OS 6_0 like Mac
OS X) AppleWebKit/536.26 (KHTML, like Gecko) Version/6.0 Mobile/10A5376e Safari
/8536.25 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
86.1.76.62 - - [04/Jan/2015:05:30:37 +0000] "GET /projects/xdotool/ HTTP/1.1" 20
0 12292 "http://www.haskell.org/haskellwiki/Xmonad/Frequently_asked_questions" "
Mozilla/5.0 (X11; Linux x86_64; rv:24.0) Gecko/20140205 Firefox/24.0 Iceweasel/2
4.3.0"
86.1.76.62 - - [04/Jan/2015:05:30:37 +0000] "GET /reset.css HTTP/1.1" 200 1015 "
http://www.semicomplete.com/projects/xdotool/" "Mozilla/5.0 (X11; Linux x86_64;
rv:24.0) Gecko/20140205 Firefox/24.0 Iceweasel/24.3.0"
86.1.76.62 - - [04/Jan/2015:05:30:37 +0000] "GET /style2.css HTTP/1.1" 200 4877
"http://www.semicomplete.com/projects/xdotool/" "Mozilla/5.0 (X11; Linux x86_64;
rv:24.0) Gecko/20140205 Firefox/24.0 Iceweasel/24.3.0"
[analyst@secOps ~]$
[analyst@secOps ~]$
[analyst@secOps ~]$
[analyst@secOps ~]$
[analyst@secOps ~]$
[analyst@secOps ~]$
[analyst@secOps ~]$ less /home/analyst/lab.support.files/logstash-tutorial.log
[analyst@secOps ~]$
```

- e. La commande **tail** affiche la fin d'un fichier texte. Par défaut, **tail** affiche les dix dernières lignes du fichier.

Utilisez **tail** pour afficher les dix dernières lignes du fichier **/home/analyst/lab.support.files/logstash-tutorial.log**.

```
analyst@secOps ~$ tail /home/analyst/lab.support.files/logstash-tutorial.log
```

Capture d'écran de l'affichage après l'exécution de la commande « **tail** » :



```

Terminal
File Edit View Search Terminal Help

[analyst@secOps ~]$ tail /home/analyst/lab.support.files/logstash-tutorial.log
218.30.103.62 - - [04/Jan/2015:05:28:43 +0000] "GET /blog/geekery/xvfb-firefox.html HTTP/1.1" 200 10975 "-" "Sogou web spider/4.0(+http://www.sogou.com/docs/help/webmasters.htm#07)"
218.30.103.62 - - [04/Jan/2015:05:29:06 +0000] "GET /blog/geekery/puppet-facts-into-mcollective.html HTTP/1.1" 200 9872 "-" "Sogou web spider/4.0(+http://www.sogou.com/docs/help/webmasters.htm#07)"
198.46.149.143 - - [04/Jan/2015:05:29:13 +0000] "GET /blog/geekery/disabling-battery-in-ubuntu-vms.html?utm_source=feedburner&utm_medium=feed&utm_campaign=Feed%3A+semicomplete%2Fmain+%28semicomplete.com+-+Jordan+Sissel%29 HTTP/1.1" 200 9316 "-" "Tiny Tiny RSS/1.11 (http://tt-rss.org/)"
198.46.149.143 - - [04/Jan/2015:05:29:13 +0000] "GET /blog/geekery/solving-good-or-bad-problems.html?utm_source=feedburner&utm_medium=feed&utm_campaign=Feed%3A+semicomplete%2Fmain+%28semicomplete.com+-+Jordan+Sissel%29 HTTP/1.1" 200 10756 "-" "Tiny Tiny RSS/1.11 (http://tt-rss.org/)"
218.30.103.62 - - [04/Jan/2015:05:29:26 +0000] "GET /blog/geekery/jquery-interface-puffer.html%20target= HTTP/1.1" 200 202 "-" "Sogou web spider/4.0(+http://www.sogou.com/docs/help/webmasters.htm#07)"
218.30.103.62 - - [04/Jan/2015:05:29:48 +0000] "GET /blog/geekery/ec2-reserved-vs-ondemand.html HTTP/1.1" 200 11834 "-" "Sogou web spider/4.0(+http://www.sogou.com/docs/help/webmasters.htm#07)"
66.249.73.135 - - [04/Jan/2015:05:30:06 +0000] "GET /blog/web/firefox-scrolling-fix.html HTTP/1.1" 200 8956 "-" "Mozilla/5.0 (iPhone; CPU iPhone OS 6_0 like Mac OS X) AppleWebKit/536.26 (KHTML, like Gecko) Version/6.0 Mobile/10A5376e Safari/8536.25 (compatible; Googlebot/2.1; +http://www.google.com/bot.html)"
86.1.76.62 - - [04/Jan/2015:05:30:37 +0000] "GET /projects/xdotool/ HTTP/1.1" 200 12292 "http://www.haskell.org/haskellwiki/Xmonad/Frequently_asked_questions" "Mozilla/5.0 (X11; Linux x86_64; rv:24.0) Gecko/20140205 Firefox/24.0 Iceweasel/24.3.0"
86.1.76.62 - - [04/Jan/2015:05:30:37 +0000] "GET /reset.css HTTP/1.1" 200 1015 "http://www.semicomplete.com/projects/xdotool/" "Mozilla/5.0 (X11; Linux x86_64; rv:24.0) Gecko/20140205 Firefox/24.0 Iceweasel/24.3.0"
86.1.76.62 - - [04/Jan/2015:05:30:37 +0000] "GET /style2.css HTTP/1.1" 200 4877 "http://www.semicomplete.com/projects/xdotool/" "Mozilla/5.0 (X11; Linux x86_64; rv:24.0) Gecko/20140205 Firefox/24.0 Iceweasel/24.3.0"
[analyst@secOps ~]$

```

Étape 2: Suivre activement les logs.

Dans certains cas, il est utile de consulter les fichiers journaux à mesure que des entrées y sont ajoutées. Dans ce cas, la commande **tail -f** est très utile.

- a. Utiliser **tail -f** pour surveiller activement le contenu du fichier **/var/log/syslog**:

```
analyst@secOps ~$ sudo tail -f
/home/analyst/lab.support.files/logstash-tutorial.log
```

Quelle est la différence dans la sortie de **tail** et **tail -f**? Expliquez votre réponse.

1. Commande **tail** (Statique) la commande **tail** (sans option) :

Affiche par défaut les dix dernières lignes du fichier au moment où vous l'exécutez. Elle se termine immédiatement après avoir affiché ces lignes. La sortie est statique : elle ne changera pas, même si d'autres applications ajoutent de nouvelles lignes au fichier.

2. Commande **tail -f** (Dynamique/Suivi) la commande **tail -f** (où **-f** signifie "suivre") :

Affiche d'abord les dix dernières lignes du fichier. Elle reste active et surveille le fichier en continu. Elle affiche en temps réel toute nouvelle ligne ajoutée à la fin du fichier. La sortie est dynamique. C'est idéal pour suivre les journaux en direct pendant qu'une application fonctionne ou lors d'un dépannage. Pour l'arrêter et revenir à l'invite du shell, il suffit d'appuyer sur CTRL+C.

- b. Pour observer **tail -f** en action, ouvrez un second terminal. Organisez votre écran de manière à voir les deux fenêtres du terminal. Redimensionner les fenêtres pour afficher les deux en même temps, comme indiqué dans l'image ci-dessous:

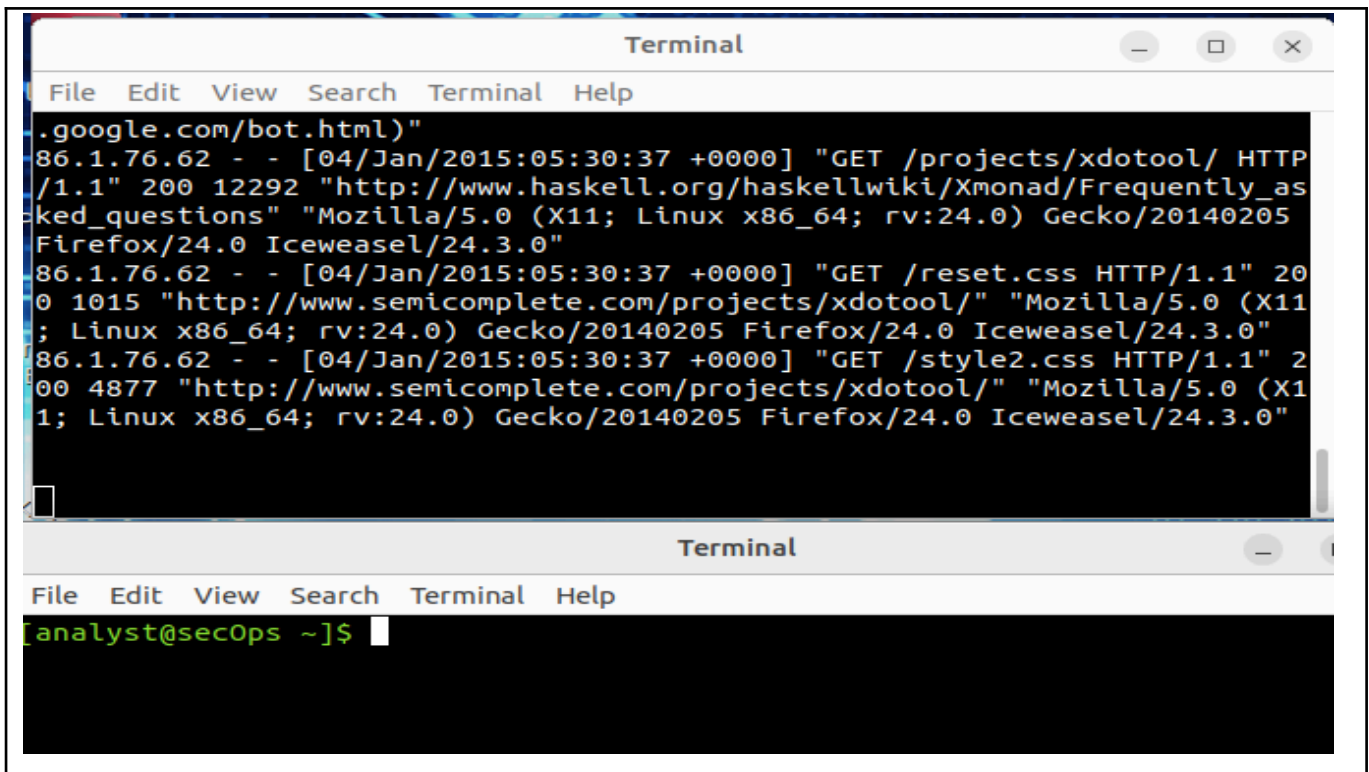
La fenêtre de terminal du haut exécute **tail -f** pour surveiller le fichier :

/home/analyst/lab.support.files/logstash-tutorial.log.

Utilisez la fenêtre du bas pour ajouter des informations sur le fichier surveillé.

Pour faciliter la visualisation, sélectionnez la fenêtre du terminal du haut (celle qui exécute **tail -f**) et appuyez plusieurs fois sur la touche Entrée. Cela insérera quelques lignes entre le contenu actuel du fichier et les nouvelles informations à ajouter.

Capture d'écran de l'affichage après l'exécution de la commande « tail -f » et redimensionnement des fenêtres :



```

Terminal
File Edit View Search Terminal Help
.google.com/bot.html)"
86.1.76.62 - - [04/Jan/2015:05:30:37 +0000] "GET /projects/xdotool/ HTTP
/1.1" 200 12292 "http://www.haskell.org/haskellwiki/Xmonad/Frequently_as
ked_questions" "Mozilla/5.0 (X11; Linux x86_64; rv:24.0) Gecko/20140205
Firefox/24.0 Iceweasel/24.3.0"
86.1.76.62 - - [04/Jan/2015:05:30:37 +0000] "GET /reset.css HTTP/1.1" 20
0 1015 "http://www.semicomplete.com/projects/xdotool/" "Mozilla/5.0 (X11
; Linux x86_64; rv:24.0) Gecko/20140205 Firefox/24.0 Iceweasel/24.3.0"
86.1.76.62 - - [04/Jan/2015:05:30:37 +0000] "GET /style2.css HTTP/1.1" 2
00 4877 "http://www.semicomplete.com/projects/xdotool/" "Mozilla/5.0 (X1
1; Linux x86_64; rv:24.0) Gecko/20140205 Firefox/24.0 Iceweasel/24.3.0"

Terminal
File Edit View Search Terminal Help
[analyst@secOps ~]$

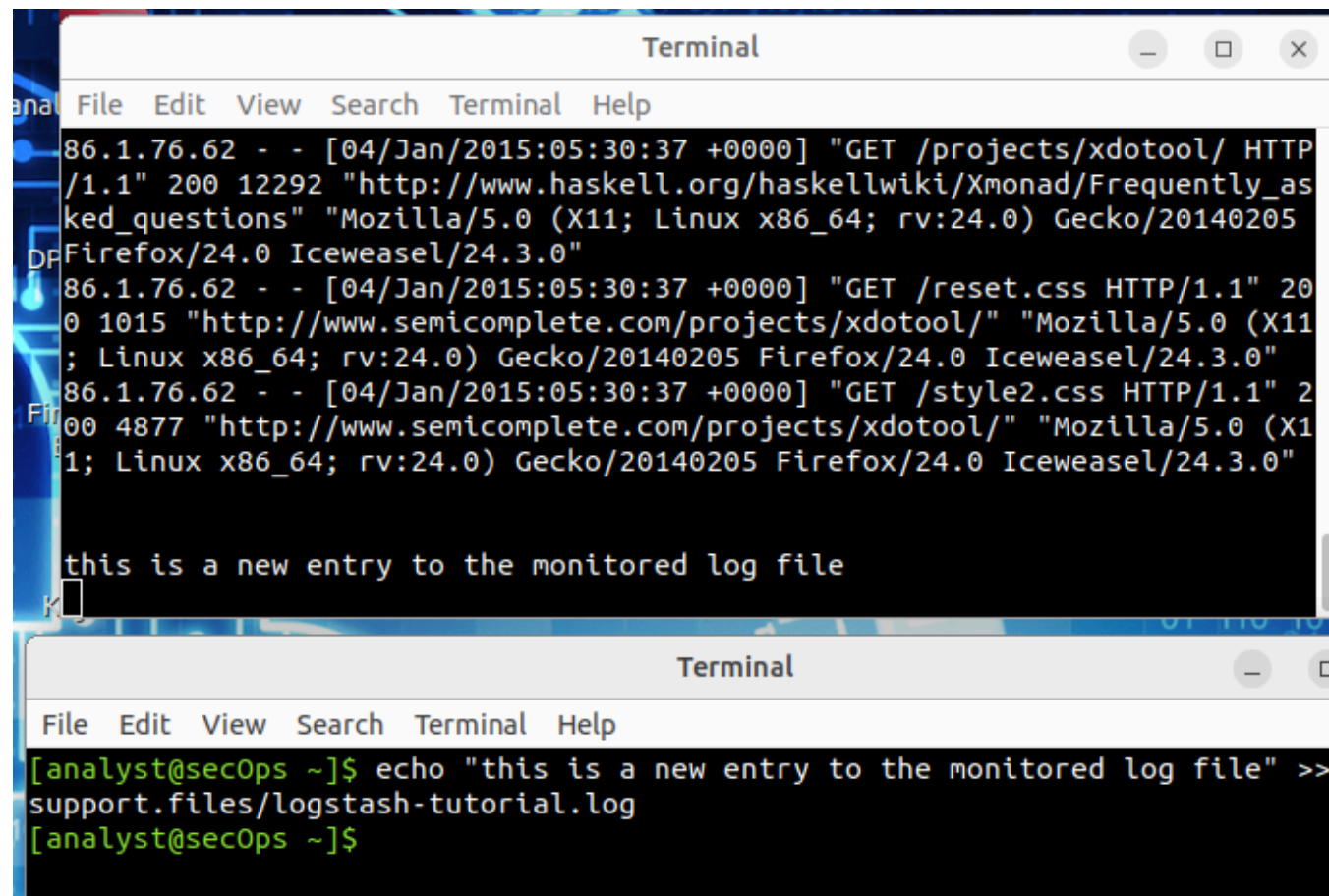
```


- c. Sélectionnez la fenêtre inférieure du terminal et entrez la commande suivante:

```
[analyst@secOps ~]$ echo "this is a new entry to the monitored log file" >>
lab.support.files/logstash-tutorial.log
```

La commande ci-dessus ajoute le message "this is a new entry to the monitored log file" au fichier **/home/analyst/lab.support.files/logstash-tutorial.log**. Comme la commande **tail -f** surveille actuellement le fichier, une ligne est ajoutée au fichier. La fenêtre du haut devrait afficher la nouvelle ligne en temps réel.

Capture d'écran de l'affichage avec la nouvelle ligne en temps réel :



Validation :

- d. Appuyez sur CTRL+C pour arrêter l'exécution de **tail -f** et revenir à l'invite du shell.
e. Fermez l'une des deux fenêtres du terminal.

Partie 2: Fichiers journaux et Syslog

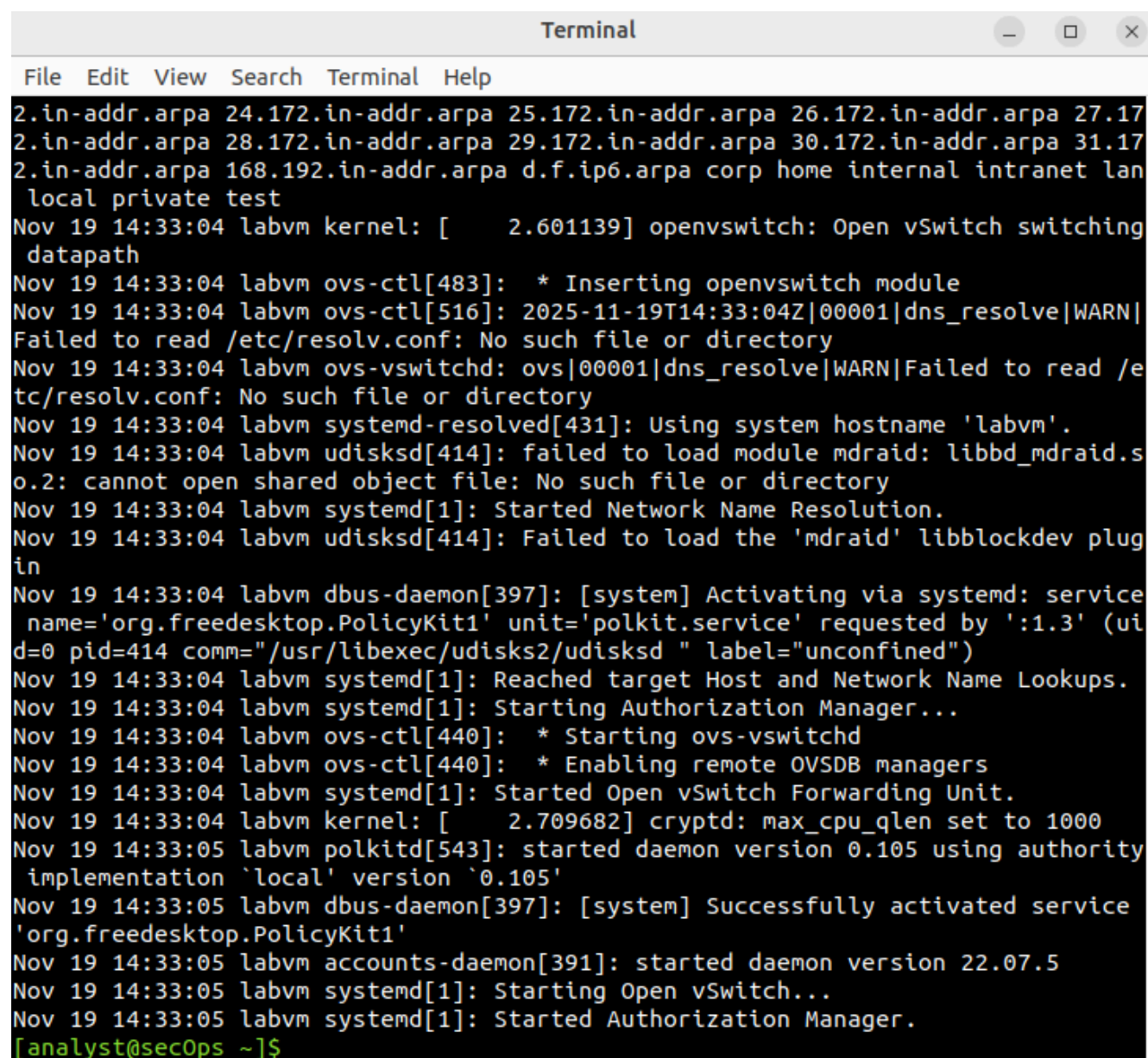
En raison de leur grande utilité, les fichiers journaux sont généralement stockés sur le même ordinateur de surveillance. **Syslog** est un système conçu pour permettre aux périphériques d'envoyer leurs fichiers journaux à un serveur centralisé, appelé serveur **syslog**. Les clients communiquent avec un serveur syslog en utilisant le protocole **syslog**. **Syslog** est couramment déployé et supporte pratiquement toutes les plateformes informatiques.

La VM Security Workstation génère des fichiers journaux au niveau du système d'exploitation et les transmet à **syslog**.

- a. Utilisez la commande **cat** en tant que **root** pour lister le contenu du fichier **/var/log/syslog.1**. Ce fichier contient les entrées de journal générées par le système d'exploitation Security Workstation VM et envoyées au service **syslog**.

```
analyst@secOps ~$ sudo cat /var/log/syslog.1
```

Capture d'écran de l'affichage après l'exécution de la commande « cat » :



```
Terminal
File Edit View Search Terminal Help
2.in-addr.arpa 24.172.in-addr.arpa 25.172.in-addr.arpa 26.172.in-addr.arpa 27.17
2.in-addr.arpa 28.172.in-addr.arpa 29.172.in-addr.arpa 30.172.in-addr.arpa 31.17
2.in-addr.arpa 168.192.in-addr.arpa d.f.ip6.arpa corp home internal intranet lan
local private test
Nov 19 14:33:04 labvm kernel: [ 2.601139] openvswitch: Open vSwitch switching
datapath
Nov 19 14:33:04 labvm ovs-ctl[483]: * Inserting openvswitch module
Nov 19 14:33:04 labvm ovs-ctl[516]: 2025-11-19T14:33:04Z|00001|dns_resolve|WARN|
Failed to read /etc/resolv.conf: No such file or directory
Nov 19 14:33:04 labvm ovs-vswitchd: ovs|00001|dns_resolve|WARN|Failed to read /e
tc/resolv.conf: No such file or directory
Nov 19 14:33:04 labvm systemd-resolved[431]: Using system hostname 'labvm'.
Nov 19 14:33:04 labvm udisksd[414]: failed to load module mdraid: libbd_mdraid.s
o.2: cannot open shared object file: No such file or directory
Nov 19 14:33:04 labvm systemd[1]: Started Network Name Resolution.
Nov 19 14:33:04 labvm udisksd[414]: Failed to load the 'mdraid' libblockdev plug
in
Nov 19 14:33:04 labvm dbus-daemon[397]: [system] Activating via systemd: service
name='org.freedesktop.PolicyKit1' unit='polkit.service' requested by ':1.3' (ui
d=0 pid=414 comm="/usr/libexec/udisks2/udisksd " label="unconfined")
Nov 19 14:33:04 labvm systemd[1]: Reached target Host and Network Name Lookups.
Nov 19 14:33:04 labvm systemd[1]: Starting Authorization Manager...
Nov 19 14:33:04 labvm ovs-ctl[440]: * Starting ovs-vswitchd
Nov 19 14:33:04 labvm ovs-ctl[440]: * Enabling remote OVSDB managers
Nov 19 14:33:04 labvm systemd[1]: Started Open vSwitch Forwarding Unit.
Nov 19 14:33:04 labvm kernel: [ 2.709682] cryptd: max_cpu_qlen set to 1000
Nov 19 14:33:05 labvm polkitd[543]: started daemon version 0.105 using authority
implementation 'local' version '0.105'
Nov 19 14:33:05 labvm dbus-daemon[397]: [system] Successfully activated service
'org.freedesktop.PolicyKit1'
Nov 19 14:33:05 labvm accounts-daemon[391]: started daemon version 22.07.5
Nov 19 14:33:05 labvm systemd[1]: Starting Open vSwitch...
Nov 19 14:33:05 labvm systemd[1]: Started Authorization Manager.
[analyst@secOps ~]$
```


Pourquoi la commande **cat** doit-elle être exécutée en tant que **root** ?

- Pour pouvoir utiliser la commande **cat** (ou tout autre outil de lecture), il est essentiel de l'exécuter avec les privilèges de l'utilisateur **root** (ou via **sudo**). Cela est nécessaire pour accéder à des fichiers comme **/var/log/syslog.1**, et ce, pour des raisons de sécurité et de gestion des permissions dans le système d'exploitation Linux.
- Permissions Restreintes : Les fichiers journaux du système, qui se trouvent dans des répertoires sensibles comme **/var/log/**, contiennent des informations cruciales (comme des erreurs système, des activités des services, des tentatives de connexion, etc.). Par défaut, le système d'exploitation configure ces fichiers pour qu'ils ne soient lisibles que par l'utilisateur **root** et quelques groupes spécifiques (comme **adm** ou **syslog**).
- Protection du Système : Cette restriction empêche un utilisateur non privilégié (comme un analyste dans ce labo) d'accéder ou de modifier ces journaux. Cela permet de garantir l'intégrité et la confidentialité des données de journalisation.
- Accès Requis : Quand vous tapez **sudo cat /var/log/syslog.1**, vous demandez au système d'exécuter la commande **cat** avec les droits temporaires de l'utilisateur **root**. Cela lui permet de contourner la restriction de lecture et d'afficher le contenu du fichier.

- b. Notez que le fichier **/var/log/syslog** ne stocke que les entrées de journal les plus récentes. Pour que le fichier syslog reste petit, le système d'exploitation fait périodiquement tourner les fichiers journaux, en renommant les anciens fichiers journaux en **syslog.1**, **syslog.2** et ainsi de suite.

Utilisez la commande **cat** pour lister les anciens fichiers **syslog**:

```
analyst@secOps ~$ sudo cat /var/log/syslog.2
```

```
analyst@secOps ~$ sudo cat /var/log/syslog.3
```

```
analyst@secOps ~$ sudo cat /var/log/syslog.4
```

Capture d'écran de l'affichage après l'exécution de la commande « cat » pour syslog.2 :

```
[analyst@secOps ~]$ sudo cat /var/log/syslog.2
cat: /var/log/syslog.2: No such file or directory
[analyst@secOps ~]$
```

Capture d'écran de l'affichage après l'exécution de la commande « cat » pour syslog.3 :

```
[analyst@secOps ~]$ sudo cat /var/log/syslog.3
cat: /var/log/syslog.3: No such file or directory
[analyst@secOps ~]$
```

Capture d'écran de l'affichage après l'exécution de la commande « cat » pour syslog.4 :

```
[analyst@secOps ~]$ sudo cat /var/log/syslog.4
cat: /var/log/syslog.4: No such file or directory
[analyst@secOps ~]$
```

D'après vous, pourquoi est-il si important de toujours synchroniser l'heure et la date des ordinateurs?

Les journaux (logs) permettent d'avoir connaissance des événements intervenus en temps opportun ! Alors, quand survient un incident (un bug, une intrusion), les journaux enregistrent tout. Mais s'il est 10h sur le Serveur A et 10h05 sur le Serveur B, on ne peut pas relier les événements et en tirer des conclusions. Enquête chaotique !

La sécurité est rompue si l'heure est fausse. Les systèmes de sécurité, beaucoup sont basés sur la bonne heure (les certificats, les mots de passe temporaires). Si l'heure est calée sur faux, le système suppose que votre certificat est périmé ou que votre code de double authentification est faux, donc impossible de se connecter.

Les tâches planifiées. Si la sauvegarde est programmée à minuit, il faut que l'ordinateur ait bien connaissance de l'heure pour la déclencher à temps.

Validation :

Partie 3: Fichiers journaux et Journalctl

Un autre système populaire de gestion des journaux est connu sous le nom de **journal**. Géré par le démon **journald**, ce système est conçu pour centraliser la gestion des journaux, quelle que soit l'origine des messages. Dans le contexte de ce labo, la caractéristique la plus évidente du démon **journald** est l'utilisation de fichiers binaires append-only servant de **fichiers journaux**.

Étape 1: Exécuter journalctl sans option

- Pour consulter les journaux de **journald**, utilisez la commande **journalctl**. L'outil **journalctl** interprète et affiche les entrées de journal précédemment stockées dans les fichiers journaux binaires du **journal**.

```
analyst@secOps ~$ journalctl
```

Capture d'écran de l'affichage après l'exécution de la commande « journalctl » :

```

Terminal
File Edit View Search Terminal Help
Nov 19 13:35:34 labvm dbus-daemon[1163]: [session uid=1002 pid=1163] Successful>
Nov 19 13:35:34 labvm spice-vdagent[1543]: vdagent virtio channel /dev/virtio-p>
Nov 19 13:35:34 labvm brisk-menu[1511]: gdk_window_get_origin: assertion 'GDK_I>
Nov 19 13:35:34 labvm brisk-menu[1511]: gdk_window_get_origin: assertion 'GDK_I>
Nov 19 13:35:34 labvm brisk-menu[1511]: Negative content width -1 (allocation 1>
Nov 19 13:35:34 labvm brisk-menu[1511]: org.mate.ScreenSaver not running: GDBus>
Nov 19 13:35:35 labvm wnck-applet[1508]: Negative content width -1 (allocation >
Nov 19 13:35:35 labvm wnck-applet[1508]: Negative content width -1 (allocation >
Nov 19 13:35:35 labvm dbus-daemon[1163]: [session uid=1002 pid=1163] Activating>
Nov 19 13:35:35 labvm systemd[1141]: Starting Virtual filesystem metadata servi>
Nov 19 13:35:35 labvm dbus-daemon[1163]: [session uid=1002 pid=1163] Successful>
Nov 19 13:35:35 labvm systemd[1141]: Started Virtual filesystem metadata servic>
Nov 19 13:47:09 labvm sudo[2099]: analyst : TTY=pts/0 ; PWD=/home/analyst ; US>
Nov 19 13:47:09 labvm sudo[2099]: pam_unix(sudo:session): session opened for us>
Nov 19 13:47:42 labvm sudo[2099]: pam_unix(sudo:session): session closed for us>
Nov 19 13:49:27 labvm sudo[2152]: analyst : TTY=pts/0 ; PWD=/home/analyst ; US>
Nov 19 13:49:27 labvm sudo[2152]: pam_unix(sudo:session): session opened for us>
Nov 19 13:54:31 labvm sudo[2152]: pam_unix(sudo:session): session closed for us>
Nov 19 13:54:54 labvm wnck-applet[1508]: Negative content width -1 (allocation >
Nov 19 13:54:54 labvm wnck-applet[1508]: Negative content width -1 (allocation >
Nov 19 13:55:25 labvm sudo[2235]: analyst : TTY=pts/0 ; PWD=/home/analyst ; US>
Nov 19 13:55:25 labvm sudo[2235]: pam_unix(sudo:session): session opened for us>
Nov 19 13:55:25 labvm sudo[2235]: pam_unix(sudo:session): session closed for us>
Nov 19 13:56:48 labvm sudo[2264]: analyst : TTY=pts/0 ; PWD=/home/analyst ; US>
Nov 19 13:56:48 labvm sudo[2264]: pam_unix(sudo:session): session opened for us>
Nov 19 13:56:48 labvm sudo[2264]: pam_unix(sudo:session): session closed for us>
Nov 19 13:57:17 labvm sudo[2289]: analyst : TTY=pts/0 ; PWD=/home/analyst ; US>
Nov 19 13:57:17 labvm sudo[2289]: pam_unix(sudo:session): session opened for us>
Nov 19 13:57:17 labvm sudo[2289]: pam_unix(sudo:session): session closed for us>
Nov 19 13:57:31 labvm sudo[2294]: analyst : TTY=pts/0 ; PWD=/home/analyst ; US>
Nov 19 13:57:31 labvm sudo[2294]: pam_unix(sudo:session): session opened for us>
Nov 19 13:57:31 labvm sudo[2294]: pam_unix(sudo:session): session closed for us>
[analyst@secOps ~]$

```

Remarque: L'exécution de journalctl en tant que root affichera des informations plus détaillées.

- Utilisez CTRL+C pour quitter l'affichage.

Étape 2: Journalctl et quelques options

Une partie de la puissance de l'utilisation de **journalctl** réside dans ses options. Pour les commandes suivantes, utilisez CTRL+C pour quitter l'écran.

- Utilisez **journalctl --utc** pour afficher tous les horodatages en temps UTC:

```
analyst@secOps ~$ sudo journalctl --utc
```

Capture d'écran de l'affichage après l'exécution de la commande « **journalctl** avec l'option **utc** » :

```

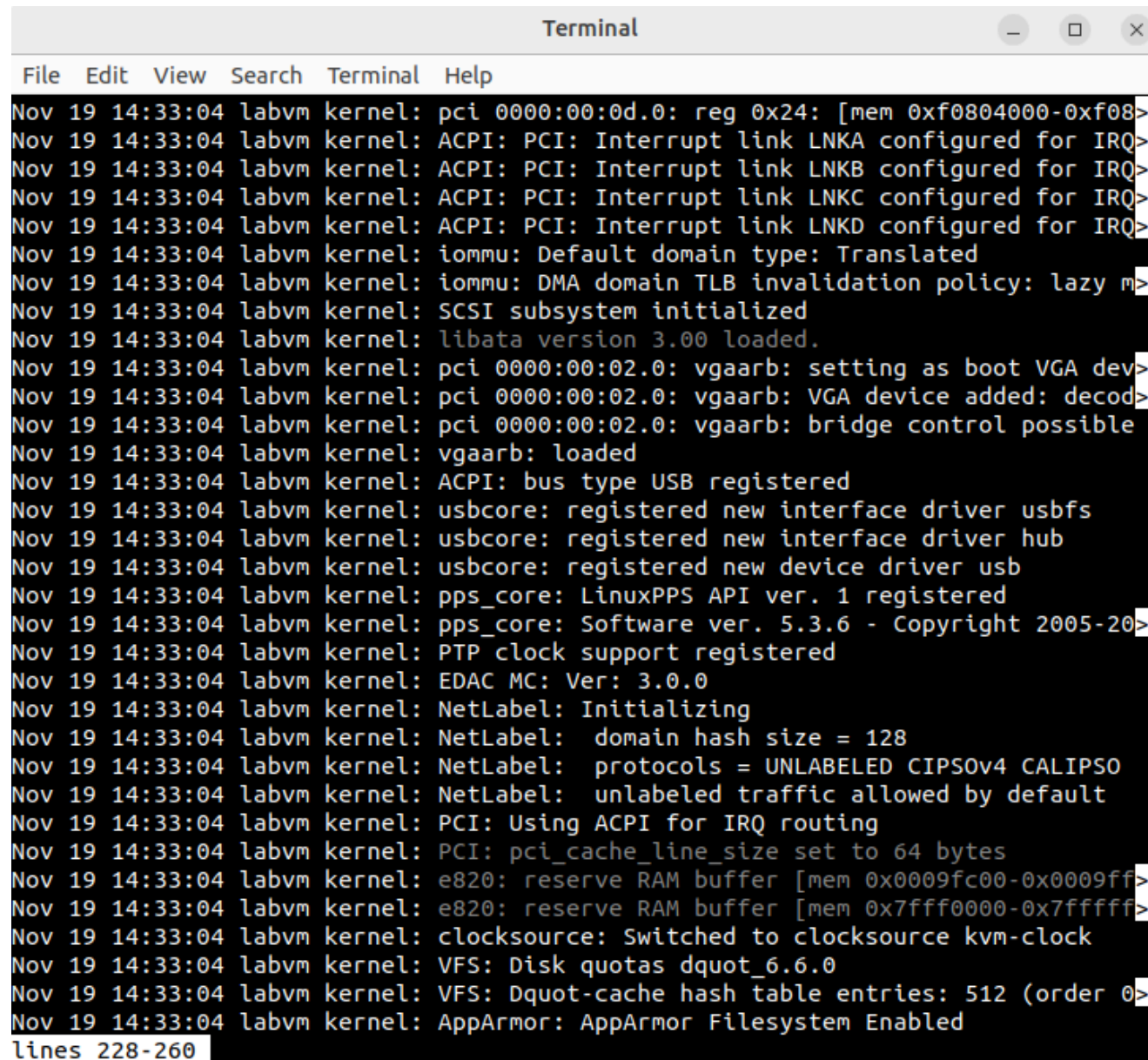
Terminal
File Edit View Search Terminal Help
Feb 10 21:18:38 labvm systemd[1]: cloud-init-hotplugd.socket: Socket unit confi>
Feb 10 21:18:38 labvm systemd[1]: Reloading.
Feb 10 21:18:38 labvm systemd[1]: cloud-init-hotplugd.socket: Socket unit confi>
Feb 10 21:18:38 labvm systemd[1]: Started Download data for packages that fail>
Feb 10 21:18:38 labvm systemd[1]: Starting Download data for packages that fail>
Feb 10 21:18:38 labvm systemd[1]: Started Check to see whether there is a new v>
Feb 10 21:18:38 labvm systemd[1]: update-notifier-download.service: Deactivated>
Feb 10 21:18:38 labvm systemd[1]: Finished Download data for packages that fail>
Feb 10 21:18:40 labvm systemd[1392]: Started D-Bus User Message Bus.
Feb 10 21:18:40 labvm dbus-daemon[22443]: [session uid=1000 pid=22443] AppArmor>
Feb 10 21:18:47 labvm dbus-daemon[814]: [system] Reloaded configuration
Feb 10 21:18:49 labvm dbus-daemon[814]: [system] Reloaded configuration
Feb 10 21:19:20 labvm systemd[1]: Reloading.
Feb 10 21:19:20 labvm systemd[1]: cloud-init-hotplugd.socket: Socket unit confi>
Feb 10 21:19:20 labvm systemd[1]: Reloading.
Feb 10 21:19:20 labvm systemd[1]: cloud-init-hotplugd.socket: Socket unit confi>
Feb 10 21:19:20 labvm systemd[1]: Reloading.
Feb 10 21:19:20 labvm systemd[1]: cloud-init-hotplugd.socket: Socket unit confi>
Feb 10 21:19:26 labvm dbus-daemon[814]: Unknown username "pulse" in message bus>
Feb 10 21:19:26 labvm dbus-daemon[814]: [system] Reloaded configuration
Feb 10 21:19:26 labvm groupadd[28751]: group added to /etc/group: name=pulse, G>
Feb 10 21:19:26 labvm groupadd[28751]: group added to /etc/gshadow: name=pulse
Feb 10 21:19:26 labvm groupadd[28751]: new group: name=pulse, GID=121
Feb 10 21:19:26 labvm useradd[28757]: new user: name=pulse, UID=114, GID=121, h>
Feb 10 21:19:26 labvm usermod[28764]: change user 'pulse' password
Feb 10 21:19:26 labvm chage[28771]: changed password expiry for pulse
Feb 10 21:19:26 labvm chfn[28775]: changed user 'pulse' information
Feb 10 21:19:26 labvm gpasswd[28782]: user pulse added by root to group audio
Feb 10 21:19:26 labvm groupadd[28790]: group added to /etc/group: name=pulse-ac>
Feb 10 21:19:26 labvm groupadd[28790]: group added to /etc/gshadow: name=pulse->
Feb 10 21:19:26 labvm groupadd[28790]: new group: name=pulse-access, GID=122
Feb 10 21:19:27 labvm systemd[1]: Reloading.
Feb 10 21:19:27 labvm systemd[1]: cloud-init-hotplugd.socket: Socket unit confi>
lines 1512-1544

```


- b. Utilisez **journalctl -b** pour afficher les entrées du journal enregistrées lors du dernier démarrage:

```
analyst@secOps ~$ sudo journalctl -b
```

Capture d'écran de l'affichage après l'exécution de la commande « journalctl avec l'option b » :



```

Terminal
File Edit View Search Terminal Help
Nov 19 14:33:04 labvm kernel: pci 0000:00:0d.0: reg 0x24: [mem 0xf0804000-0xf08>
Nov 19 14:33:04 labvm kernel: ACPI: PCI: Interrupt link LNKA configured for IRQ>
Nov 19 14:33:04 labvm kernel: ACPI: PCI: Interrupt link LNKB configured for IRQ>
Nov 19 14:33:04 labvm kernel: ACPI: PCI: Interrupt link LNKC configured for IRQ>
Nov 19 14:33:04 labvm kernel: ACPI: PCI: Interrupt link LNKD configured for IRQ>
Nov 19 14:33:04 labvm kernel: iommu: Default domain type: Translated
Nov 19 14:33:04 labvm kernel: iommu: DMA domain TLB invalidation policy: lazy m>
Nov 19 14:33:04 labvm kernel: SCSI subsystem initialized
Nov 19 14:33:04 labvm kernel: libata version 3.00 loaded.
Nov 19 14:33:04 labvm kernel: pci 0000:00:02.0: vgaarb: setting as boot VGA dev>
Nov 19 14:33:04 labvm kernel: pci 0000:00:02.0: vgaarb: VGA device added: decod>
Nov 19 14:33:04 labvm kernel: pci 0000:00:02.0: vgaarb: bridge control possible
Nov 19 14:33:04 labvm kernel: vgaarb: loaded
Nov 19 14:33:04 labvm kernel: ACPI: bus type USB registered
Nov 19 14:33:04 labvm kernel: usbcore: registered new interface driver usbfs
Nov 19 14:33:04 labvm kernel: usbcore: registered new interface driver hub
Nov 19 14:33:04 labvm kernel: usbcore: registered new device driver usb
Nov 19 14:33:04 labvm kernel: pps_core: LinuxPPS API ver. 1 registered
Nov 19 14:33:04 labvm kernel: pps_core: Software ver. 5.3.6 - Copyright 2005-20>
Nov 19 14:33:04 labvm kernel: PTP clock support registered
Nov 19 14:33:04 labvm kernel: EDAC MC: Ver: 3.0.0
Nov 19 14:33:04 labvm kernel: NetLabel: Initializing
Nov 19 14:33:04 labvm kernel: NetLabel: domain hash size = 128
Nov 19 14:33:04 labvm kernel: NetLabel: protocols = UNLABELED CIPSOv4 CALIPSO
Nov 19 14:33:04 labvm kernel: NetLabel: unlabeled traffic allowed by default
Nov 19 14:33:04 labvm kernel: PCI: Using ACPI for IRQ routing
Nov 19 14:33:04 labvm kernel: PCI: pci_cache_line_size set to 64 bytes
Nov 19 14:33:04 labvm kernel: e820: reserve RAM buffer [mem 0x0009fc00-0x0009ff>
Nov 19 14:33:04 labvm kernel: e820: reserve RAM buffer [mem 0x7fff0000-0x7fffff>
Nov 19 14:33:04 labvm kernel: clocksource: Switched to clocksource kvm-clock
Nov 19 14:33:04 labvm kernel: VFS: Disk quotas dquot_6.6.0
Nov 19 14:33:04 labvm kernel: VFS: Dquot-cache hash table entries: 512 (order 0>
Nov 19 14:33:04 labvm kernel: AppArmor: AppArmor Filesystem Enabled
lines 228-260

```


- c. Utilisez **journalctl** pour spécifier le service et le délai des entrées de journal. La commande ci-dessous montre tous les journaux du service **nginx** enregistrés aujourd'hui:

```
analyst@secOps ~$ sudo journalctl -u nginx.service --since today
```

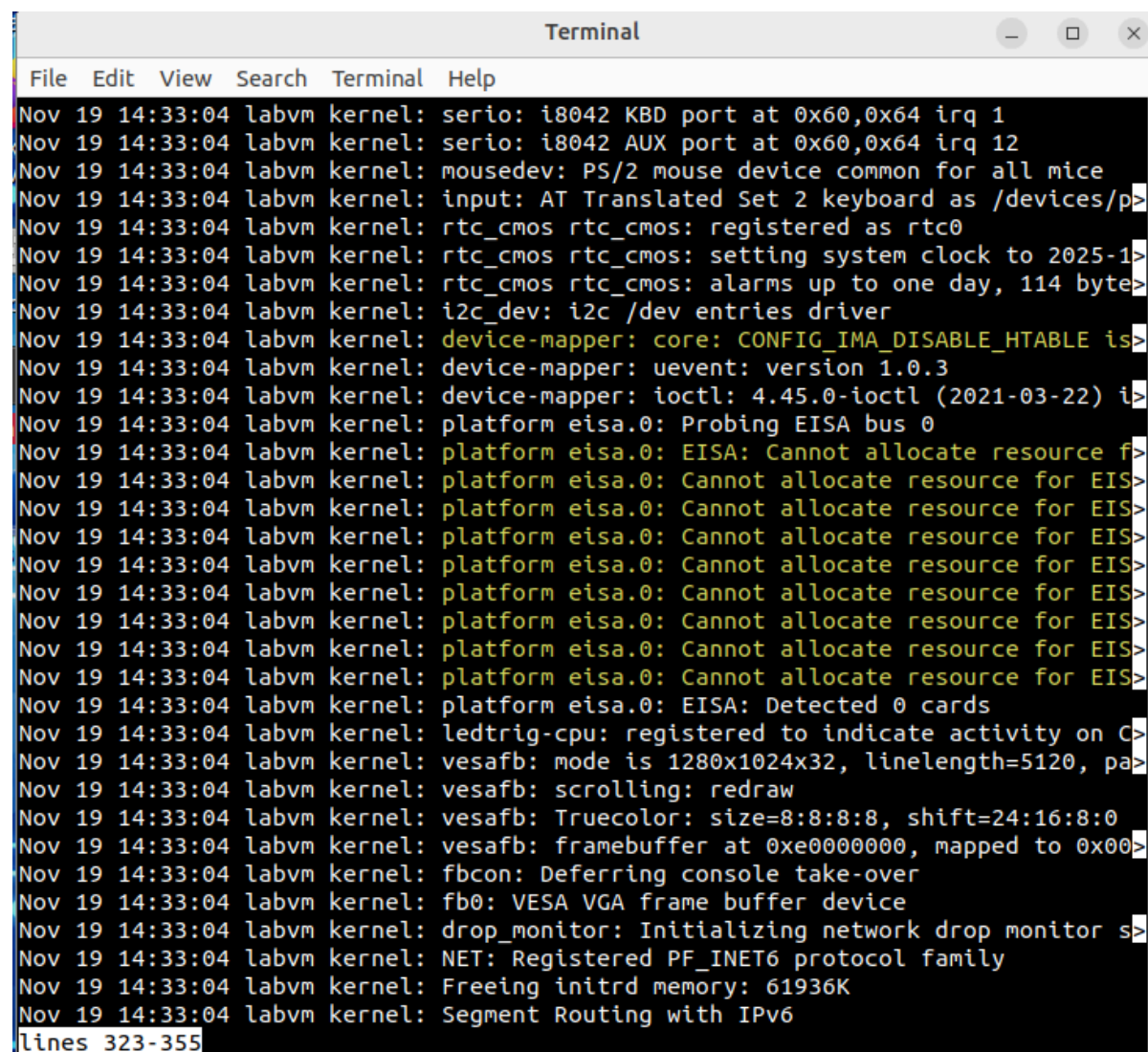
Capture d'écran de l'affichage après l'exécution de la commande « **journalctl** » :

```
[analyst@secOps ~]$ sudo journalctl -u nginx.service --since today
-- No entries --
[analyst@secOps ~]$
```

- d. Utilisez le commutateur **-k** pour afficher uniquement les messages générés par le noyau:

```
analyst@secOps ~$ sudo journalctl -k
```

Capture d'écran de l'affichage après l'exécution de la commande « **journalctl** avec le commutateur **k** » :



The screenshot shows a terminal window titled "Terminal" with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal displays a series of kernel messages from the "labvm" user, timestamped "Nov 19 14:33:04". The messages include hardware initialization details such as KBD and AUX ports, mouse device registration, RTC settings for the year 2025, i2c device entries, and EISA bus probing. Multiple "Cannot allocate resource for EISA" error messages are shown, followed by "EISA: Detected 0 cards". Other messages include LED trig-cpu registration, vesafb scrolling and framebuffer setup, fbcon deferring console take-over, VESA VGA frame buffer device initialization, network drop monitor registration, and freeing of initrd memory. The output ends with "Segment Routing with IPv6" and a line number indicator "lines 323-355".

```
Nov 19 14:33:04 labvm kernel: serio: i8042 KBD port at 0x60,0x64 irq 1
Nov 19 14:33:04 labvm kernel: serio: i8042 AUX port at 0x60,0x64 irq 12
Nov 19 14:33:04 labvm kernel: mousedev: PS/2 mouse device common for all mice
Nov 19 14:33:04 labvm kernel: input: AT Translated Set 2 keyboard as /devices/p>
Nov 19 14:33:04 labvm kernel: rtc_cmos rtc_cmos: registered as rtc0
Nov 19 14:33:04 labvm kernel: rtc_cmos rtc_cmos: setting system clock to 2025-1>
Nov 19 14:33:04 labvm kernel: rtc_cmos rtc_cmos: alarms up to one day, 114 byte>
Nov 19 14:33:04 labvm kernel: i2c_dev: i2c /dev entries driver
Nov 19 14:33:04 labvm kernel: device-mapper: core: CONFIG_IMA_DISABLE_HTABLE is>
Nov 19 14:33:04 labvm kernel: device-mapper: uevent: version 1.0.3
Nov 19 14:33:04 labvm kernel: device-mapper: ioctl: 4.45.0-ioctl (2021-03-22) i>
Nov 19 14:33:04 labvm kernel: platform eisa.0: Probing EISA bus 0
Nov 19 14:33:04 labvm kernel: platform eisa.0: EISA: Cannot allocate resource f>
Nov 19 14:33:04 labvm kernel: platform eisa.0: Cannot allocate resource for EIS>
Nov 19 14:33:04 labvm kernel: platform eisa.0: Cannot allocate resource for EIS>
Nov 19 14:33:04 labvm kernel: platform eisa.0: Cannot allocate resource for EIS>
Nov 19 14:33:04 labvm kernel: platform eisa.0: Cannot allocate resource for EIS>
Nov 19 14:33:04 labvm kernel: platform eisa.0: Cannot allocate resource for EIS>
Nov 19 14:33:04 labvm kernel: platform eisa.0: Cannot allocate resource for EIS>
Nov 19 14:33:04 labvm kernel: platform eisa.0: Cannot allocate resource for EIS>
Nov 19 14:33:04 labvm kernel: platform eisa.0: EISA: Detected 0 cards
Nov 19 14:33:04 labvm kernel: ledtrig-cpu: registered to indicate activity on C>
Nov 19 14:33:04 labvm kernel: vesafb: mode is 1280x1024x32, linelength=5120, pa>
Nov 19 14:33:04 labvm kernel: vesafb: scrolling: redraw
Nov 19 14:33:04 labvm kernel: vesafb: Truecolor: size=8:8:8:8, shift=24:16:8:0
Nov 19 14:33:04 labvm kernel: vesafb: framebuffer at 0xe0000000, mapped to 0x00>
Nov 19 14:33:04 labvm kernel: fbcon: Deferring console take-over
Nov 19 14:33:04 labvm kernel: fb0: VESA VGA frame buffer device
Nov 19 14:33:04 labvm kernel: drop_monitor: Initializing network drop monitor s>
Nov 19 14:33:04 labvm kernel: NET: Registered PF_INET6 protocol family
Nov 19 14:33:04 labvm kernel: Freeing initrd memory: 61936K
Nov 19 14:33:04 labvm kernel: Segment Routing with IPv6
lines 323-355
```

- e. Comme pour **tail -f** décrit ci-dessus, utilisez le commutateur **-f** pour suivre activement les journaux pendant qu'ils sont écrits:

```
analyst@secOps ~$ sudo journalctl -f
```

Capture d'écran de l'affichage après l'exécution de la commande « **journalctl** avec le commutateur **f** » :

The image shows two terminal windows. The top window displays the output of `journalctl -f`, showing system logs including service status changes and user sessions. The bottom window shows a user attempting to use `systemd-cat` with a pipe, which results in a syntax error.

```

Terminal
File Edit View Search Terminal Help
Nov 19 14:06:54 labvm systemd[1]: Started Hostname Service.
Nov 19 14:07:24 labvm systemd[1]: systemd-hostnamed.service: Deactivated successfully.
Nov 19 14:07:36 labvm sudo[2389]: pam_unix(sudo:session): session closed for user root
Nov 19 14:10:22 labvm sudo[2420]: analyst : TTY=pts/0 ; PWD=/home/analyst ; USER=root ; COMMAND=/usr/bin/journalctl -f
Nov 19 14:10:22 labvm sudo[2420]: pam_unix(sudo:session): session opened for user root(uid=0) by (uid=1002)

Nov 19 14:16:41 labvm unknown[2434]: journalctl avec le commutateur f

Terminal
File Edit View Search Terminal Help
[analyst@secOps ~]$ echo "journalctl avec le commutateur f" >> | systemd-cat
bash: syntax error near unexpected token `|'
[analyst@secOps ~]$ echo "journalctl avec le commutateur f"| systemd-cat
[analyst@secOps ~]$
  
```

Question de réflexion

Comparez les outils Syslog et Journald. Quels sont les avantages et les inconvénients de chaque outil?

Outil	Avantages (+)	Inconvénients (-)
Syslog	(+) Universel (standard pour tous les appareils). (+) Fichiers en texte clair (facile à lire).	(-) Recherche lente (nécessite grep). (-) Moins de métadonnées (moins d'infos).
Journald	(+) Recherche ultra rapide (journalctl indexé). (+) Stocke beaucoup de métadonnées (plus de contexte). (+) Meilleure intégrité (format binaire).	(-) Lié à systemd (pas universel). (-) Fichiers binaires (illisible sans journalctl).